OBJECT DRAWINGS RECAP + MORE CHALLENGING DATA STRUCTURES

Some basic facts to be remembered

In Java,

√ No variable can ever hold an object. That is, a reference variable can only hold a *reference to an object*.

√ Simply declaring a reference variable does ***not*** create an object.

√ The **`new`** operator is used to create a ***single object*** and invoke the object's constructor.

√ *Several* reference variables can refer to *the same object*. Assigning a reference will ***not*** create a new object

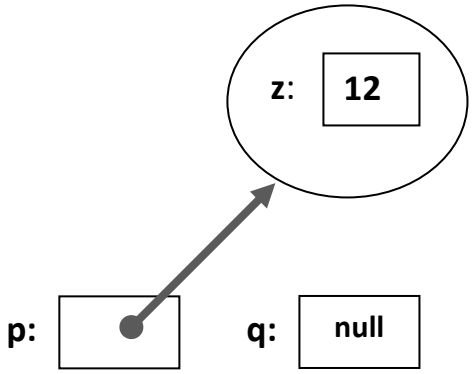
√ Arrays are objects (*instances of special built-in array classes*). When you create an array, only one object - the array object itself - is created.

The four symbols to be used in the drawings

- **VARIABLE**

*In the example, the current value of the variable **a** is **5***
*No value has been assigned to the variable **b** yet.*

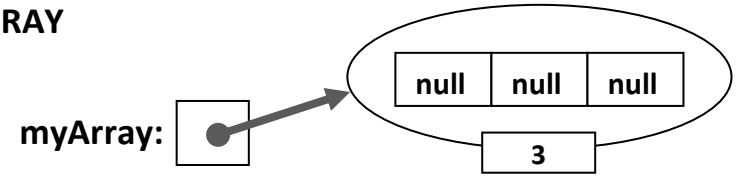
- **OBJECT**
- **REFERENCE**



*In the example, the object has one field called **z**. The current value of **z** is **12**.*

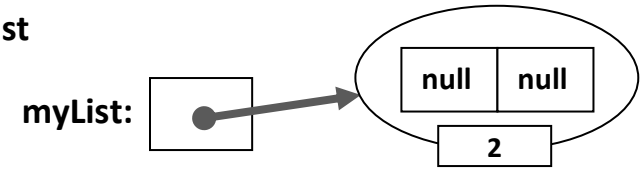
*In the example, the reference variable **p** references to the object above. The reference variable **q** does not reference to any object.*

- **ARRAY**



In the example, the array has 3 elements. The elements do not hold any references yet. The field value (3) indicates that there are 3 elements.

ArrayList



In the example, the ArrayList has 2 elements. The elements do not hold any references yet.

Excercises

1.1

```
Circle c1 = new Circle(5);
Circle c2 = new Circle(1, 2, 3);

// => Draw the variables, references, and objects NOW.
```

1.2

```
Circle c1 = new Circle(4);
Circle c2 = c1;

c1.enlarge(3);

// => Draw the variables, references, and objects NOW.
```

1.3

```
Circle r1 = new Circle(1, 2, 3);
Circle r2 = null;
Circle r3 = new Circle(3);

r3.enlarge(4);
r2 = r3;
r3 = r1;
r2.enlarge(6);

// => Draw the variables, references, and objects NOW.
```

1.4

```
Circle[] array = new Circle[3];

array[0] = new Circle(3);
array[1] = array[0];
array[2] = array[1];

for (int i = 0; i < array.length; i++) {
   array[i].enlarge(i * 6);
}

// => Draw the variables, references, and objects NOW.
```

More challenging drills for learning a little bit about OOP, data structures and algorithms ...

1.5

```
Node p = new Node(new Circle(1));

// => Draw the variables, references, and objects NOW.
```

1.6

```
Node p = new Node(new Circle(1));

p.setNext(new Node(new Circle(2)));

// => Draw the variables, references, and objects NOW.
```

1.7

```
Node first = new Node(new Circle(1));
Node p = first;

p.setNext(new Node(new Circle(2)));

p = p.getNext();
p.setNext(new Node(new Circle(3)));

// => Draw the variables, references, and objects NOW.
```

1.8

```
Node first = new Node(new Circle(1));
Node p = first;

for (int i = 2; i <= 4; i++) {
    p.setNext(new Node(new Circle(i)));
    p = p.getNext();
}

p.setNext(first);
p = null;

// => Draw the variables, references, and objects NOW.
```

1.9

```
Circle[] array = new Circle[3];

array[0] = new Circle(6);
array[1] = array[0];
array[2] = array[1];

for (Circle c : array) {
    c.enlarge(2);
}

array[0] = null;

// => Draw the variables, references, and objects NOW.
```

1.10 More advanced optional task (writing a short algorithm)

Continue from the end of the code in exercise 1.7 and write a piece of code that counts how many Circle objects there are.